

SQL: GROUP BY, Aggregation, Subqueries, Outer Joins

COSC 304 – Introduction to Database Systems



Aggregate Queries and Functions

Several queries cannot be answered using the simple form of the `SELECT` statement. These queries require a summary calculation to be performed. Examples:

- What is the maximum employee salary?
- What is the total number of hours worked on a project?
- How many employees are there in department 'D1'?

To answer these queries requires the use of aggregate functions. Aggregate functions summarize a set of rows or values and return a single value.

Aggregate Functions

Five common aggregate functions are:

- `COUNT (column)` - returns the number of non-NULL values in a column
- `SUM (column)` - returns the sum of the values in a column
- `AVG (column)` - returns the average of the values in a column
- `MIN (column)` - returns the smallest value in a column
- `MAX (column)` - returns the largest value in a column

Notes:

- 1) `COUNT` can be used with any column; `MIN` and `MAX` can be used with comparable values such as numbers, strings, and dates. `SUM` and `AVG` require numeric values.
- 2) Except for `COUNT (*)` all functions ignore nulls. `COUNT (*)` returns the number of rows in the table.
- 3) Use `DISTINCT` within an aggregate to eliminate duplicate values, e.g., `COUNT (DISTINCT title)`.

Aggregate Function Example

Return the number of employees and their average salary.

```
SELECT COUNT(eno) AS numEmp, AVG(salary) AS avgSalary  
FROM emp
```

Result

numEmp	avgSalary
8	38750

GROUP BY Clause

The **GROUP BY** clause divides rows into groups that have the same values for the specified columns.

Aggregate functions are then calculated separately for each group.

GROUP BY Example

For each employee title, return the number of employees with that title, and the minimum, maximum, and average salary.

```
SELECT  title, COUNT(*) AS numEmp,
        MIN(salary) as minSal,
        MAX(salary) as maxSal, AVG(salary) AS avgSal
FROM    emp
GROUP BY title
```

Result

title	numEmp	minSal	maxSal	avgSal
EE	2	30000	30000	30000
SA	3	50000	50000	50000
ME	2	40000	40000	40000
PR	1	20000	20000	20000

GROUP BY Clause Rules

- 1) For our queries, every column in the `SELECT` clause must either be used in an aggregate function or appear in the `GROUP BY` clause.
 - A column may appear in `GROUP BY` without appearing in `SELECT`.
- 2) `WHERE` conditions are applied before `GROUP BY` and aggregate functions are calculated.
- 3) You can group by multiple columns. Rows are in the same group only if they have the same values for all grouping columns.

GROUP BY Question

Question: Given this table and the query:

```
SELECT title, SUM(salary)
FROM emp
GROUP BY title
```

How many rows are returned?

- A) 1
- B) 2
- C) 4
- D) 8

Emp Relation

<u>eno</u>	ename	title	salary
E1	J. Doe	EE	30000
E2	M. Smith	SA	50000
E3	A. Lee	ME	40000
E4	J. Miller	PR	20000
E5	B. Casey	SA	50000
E6	L. Chu	EE	30000
E7	R. Davis	ME	40000
E8	J. Jones	SA	50000

GROUP BY Question #2

Question: Given this table and the query:

```
SELECT resp, pno, SUM(hours)
FROM workson
WHERE hours > 10
GROUP BY resp, pno
```

How many rows are returned?

A) 9 B) 7 C) 5 D) 1 E) 0

workson Table

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

HAVING Clause

The **HAVING** clause filters groups ***AFTER*** GROUP BY and aggregate functions are calculated.

Use WHERE to filter individual rows before grouping. Use HAVING to filter groups after aggregation.

HAVING conditions typically involve aggregate functions or GROUP BY columns.

- If a condition can be applied before grouping, it is usually clearer to put it in WHERE.

HAVING Example

For each title with at least 2 employees, return the title and number of employees.

```

SELECT    title, COUNT(eno) AS numEmp
FROM      emp
GROUP BY  title
HAVING    COUNT(eno) >= 2
  
```

Result

title	numEmp
EE	2
SA	3
ME	2

GROUP BY/HAVING Example

For employees born after December 1, 1965, return the average salary for each department where the average salary is > \$40,000.

```

SELECT    dname, AVG(salary) AS avgSal
FROM      emp JOIN dept ON emp.dno = dept.dno
WHERE      emp.bdate > DATE '1965-12-01'
GROUP BY  dname
HAVING     AVG(salary) > 40000
    
```

Step #1: Perform Join and Filter in WHERE clause

eno	ename	bdate	title	salary	supereno	dno	dname	mgreno
E2	M. Smith	1966-06-04	SA	50000	E5	D3	Accounting	E5
E3	A. Lee	1966-07-05	ME	40000	E7	D2	Consulting	E7
E5	B. Casey	1971-12-25	SA	50000	E8	D3	Accounting	E5
E7	R. Davis	1977-09-08	ME	40000	E8	D1	Management	E8
E8	J. Jones	1972-10-11	SA	50000	null	D1	Management	E8

GROUP BY/HAVING Example (2)

Step #2: GROUP BY on dname

eno	ename	bdate	title	salary	supereno	dno	dname	mgreno
E2	M. Smith	1966-06-04	SA	50000	E5	D3	Accounting	E5
E5	B. Casey	1971-12-25	SA	50000	E8	D3	Accounting	E5
E3	A. Lee	1966-07-05	ME	40000	E7	D2	Consulting	E7
E7	R. Davis	1977-09-08	ME	40000	E8	D1	Management	E8
E8	J. Jones	1972-10-11	SA	50000	null	D1	Management	E8



Step #3: Calculate aggregate functions

dname	avgSal
Accounting	50000
Consulting	40000
Management	45000

Step #4: Filter groups using HAVING clause

dname	avgSal
Accounting	50000
Management	45000

GROUP BY Examples

Return the average project budget:

```
SELECT AVG(budget) FROM proj
```

Return the average # of hours worked on each project:

```
SELECT pno, AVG(hours) FROM workson GROUP BY pno
```

Return the departments that have at least one project with at least two employees whose title is 'SA' working on them:

```
SELECT    DISTINCT proj.dno
FROM      proj JOIN workson ON workson.pno = proj.pno
           JOIN emp ON workson.eno=emp.eno
WHERE      emp.title = 'SA' GROUP BY proj.dno, proj.pno
HAVING     COUNT(*) >=2
```

GROUP BY/HAVING

Multi-Attribute Example



Return the employee number, department number and hours the employee worked per department where the total hours is ≥ 10 .

```
SELECT    W.eno, D.dno, SUM(hours)
FROM      workson AS W JOIN proj AS P ON W.pno = P.pno
            JOIN dept AS D ON P.dno = D.dno
GROUP BY  W.eno, D.dno
HAVING    SUM(hours)  $\geq$  10
```

Result:

eno	dno	SUM(hours)
E1	D1	12
E2	D1	24
E3	D2	48
E3	D3	10
E4	D2	18
E5	D2	24
E6	D2	48
E7	D3	36

Question:

1) How would you only return records for departments D2 and D3?

SQL Querying with GROUP BY

Question: Of the following queries, select any one which is **invalid**.

A) SELECT dname
FROM dept
GROUP BY dno

B) SELECT COUNT (*)
FROM dept

C) SELECT dno, COUNT (*)
FROM dept

D) SELECT dno, COUNT (*)
FROM dept WHERE mgreno > 'A'
GROUP BY dno, dname

Try it: SQL GROUP BY Practice Questions

Question: Write these queries:

- 1) Return the highest salary of any employee.
- 2) Return the smallest project budget.
- 3) Return the department number and average budget for its projects.
- 4) For each project that has employees working on it, return its project number, name and the total number of hours worked.
- 5) For each employee, return their number, name and the total number of hours they have worked. Only show employees with more than 30 hours.

Subqueries

A SQL query can contain other SELECT statements called subqueries. Subqueries allow the result of one query to be used by another query.

When queries are nested, the outer statement determines the contents of the final result, while the inner SELECT statements are used by the outer statement (often to lookup values for WHERE clauses).

```
SELECT      ename, salary, bdate
FROM        emp
WHERE       salary > (SELECT AVG(salary) FROM emp)
```

A subquery can be in the SELECT, FROM, WHERE or HAVING clause.

Three Types of Subqueries

Scalar subqueries - returns one column and at most one row, producing a single value. If it returns no rows, the result is `NULL`. If it returns more than one row, an error occurs.

Row subquery - returns a single row containing one or more columns.

Table subquery - returns a table containing one or more rows and columns.

Scalar Subquery Examples

Return the employees that are in the 'Accounting' department:

```
SELECT  ename
FROM    emp
WHERE    dno = (SELECT dno FROM dept
                WHERE dname = 'Accounting')
```

Return employees who work more than the average number of hours on a project assignment:

```
SELECT  ename
FROM    emp JOIN workson ON workson.eno = emp.eno
WHERE    workson.hours > (SELECT AVG(hours) FROM workson)
```

Table Subqueries

A table subquery returns a relation. There are several operators that can be used:

- `EXISTS (subquery)` - true if subquery returns at least one row
- `s IN (subquery)` - true if `s` is equal to one of the returned values
- `s > ALL (subquery)` - true if `s` is greater than **every** returned value
- `s > ANY (subquery)` - true if `s` is greater than **any** returned value

Notes:

- 1) Comparison operators such as `<`, `<=`, `=`, etc. can be used with `ANY` and `ALL`.
- 2) Common negated forms include `NOT IN` and `NOT EXISTS`.
 - Example: `s NOT IN (subquery)`

Table Subquery Examples

Return all departments who have a project with a budget greater than \$300,000:

```
SELECT dname FROM dept WHERE dno IN  
  (SELECT dno FROM proj WHERE budget > 300000)
```

Return all projects that 'J. Doe' works on:

```
SELECT pname FROM proj WHERE pno IN  
  (SELECT pno FROM workson WHERE eno =  
    (SELECT eno FROM emp WHERE ename = 'J. Doe'))
```

EXISTS Example

The EXISTS predicate is used to test whether a subquery returns any rows. EXISTS returns true if the subquery returns one or more rows.

Example: Return all employees who have the same name as someone else in the company.

```
SELECT  ename
FROM    emp as E
WHERE   EXISTS (SELECT * FROM emp as E2
                WHERE E.ename = E2.ename AND
                       E.eno <> E2.eno)
```

ANY and ALL Example

ANY - the comparison must be true for at least one value returned by the subquery.

ALL - the comparison must be true for every value returned by the subquery.

Example: Return the employees who make more than all the employees with title 'ME' make.

```
SELECT  ename
FROM    emp as E
WHERE   salary > ALL (SELECT salary FROM emp
                      WHERE title = 'ME')
```

Subquery Syntax Rules

- 1) A subquery must return the number of columns expected by the operator or comparison.
 - Example: when a single value is compared using `IN`, `ANY`, `ALL`, or a comparison operator, the subquery must return one column.

- 2) Use table aliases to clearly distinguish columns from the inner and outer queries.
 - Column names in a subquery refer to the table name in the `FROM` clause of the subquery by default.
 - A subquery that references a column from the outer query is called a correlated subquery.

Correlated Subqueries

A subquery is **correlated** if it references a column from the outer query, otherwise it is **uncorrelated**.

Conceptually, a correlated subquery is evaluated for each row considered by the outer query.

Many queries using subqueries can also be written using joins.

Do not assume that a join is always faster than a subquery. Modern query optimizers may transform subqueries into joins or use other optimization strategies.

Correlated Subquery Example

Return all employees who have the same name as another employee:

```
SELECT  ename
FROM    emp as E
WHERE    EXISTS ( SELECT eno FROM emp as E2
                   WHERE E.ename = E2.ename AND
                       E.eno <> E2.eno)
```

A join-based query:

```
SELECT DISTINCT E.ename
FROM    emp as E JOIN emp as E2 ON
        E.ename = E2.ename AND E.eno <> E2.eno
```

Types of Joins

An **equijoin** only contains the equality operator (=).

- e.g. `workson JOIN proj ON workson.pno=proj.pno`

A **natural join** is an equijoin that automatically matches *all* columns having the same names in both tables. The duplicate join columns appear only once in the result.

- e.g. `workson NATURAL JOIN proj`

Left outer join – contains all tuples of left table even if no match

Right outer join – contains all tuples of right table even if no match

Full outer join – keeps all rows from both tables. Unmatched columns contain NULL.

Specifying Joins in SQL

Types: NATURAL JOIN, FULL OUTER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, INNER JOIN, JOIN

- The keyword "outer" can be omitted for outer joins. Same with "inner".

Example: Return all departments (even those without projects) and their projects.

```
SELECT dname, pname
FROM dept LEFT OUTER JOIN proj ON dept.dno = proj.dno
```

```
SELECT dname, pname
FROM dept LEFT OUTER JOIN proj USING (dno)
```

```
SELECT dname, pname
FROM dept NATURAL LEFT JOIN proj
```

Equijoin Example

workson Table

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

proj table

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

```
SELECT *
FROM workson JOIN proj
ON workson.pno = proj.pno
```

eno	pno	resp	hours	pno	pname	budget
E1	P1	Manager	12	P1	Instruments	150000
E2	P1	Analyst	24	P1	Instruments	150000
E2	P2	Analyst	6	P2	DB Develop	135000
E3	P3	Consultant	10	P3	Budget	250000
E3	P4	Engineer	48	P4	Maintenance	310000
E4	P2	Programmer	18	P2	DB Develop	135000
E5	P2	Manager	24	P2	DB Develop	135000
E6	P4	Manager	48	P4	Maintenance	310000
E7	P3	Engineer	36	P3	Budget	250000

What is the meaning of this join?

Natural Join Example

workson Table

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

proj table

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

SELECT *
FROM workson **NATURAL JOIN** proj

eno	pno	resp	hours	pname	budget
E1	P1	Manager	12	Instruments	150000
E2	P1	Analyst	24	Instruments	150000
E2	P2	Analyst	6	DB Develop	135000
E3	P3	Consultant	10	Budget	250000
E3	P4	Engineer	48	Maintenance	310000
E4	P2	Programmer	18	DB Develop	135000
E5	P2	Manager	24	DB Develop	135000
E6	P4	Manager	48	Maintenance	310000
E7	P3	Engineer	36	Budget	250000

The natural join automatically matches the common column *pno*.

Right Outer Join Example

workson Table

eno	pno	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

proj table

pno	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

```
SELECT *
FROM workson RIGHT OUTER JOIN proj
USING (pno)
```

eno	resp	hours	pno	pname	budget
E1	Manager	12	P1	Instruments	150000
E2	Analyst	24	P1	Instruments	150000
E2	Analyst	6	P2	DB Develop	135000
E3	Consultant	10	P3	Budget	250000
E3	Engineer	48	P4	Maintenance	310000
E4	Programmer	18	P2	DB Develop	135000
E5	Manager	24	P2	DB Develop	135000
E6	Manager	48	P4	Maintenance	310000
E7	Engineer	36	P3	Budget	250000
null	null	null	P5	CAD/CAM	500000

Outer Join Question

Question: Given this table and the query:

```
SELECT *
FROM workson LEFT OUTER JOIN proj
ON workson.pno = proj.pno
```

How many rows are returned?

- A) 10
- B) 9
- C) 8
- D) 7

workson Table

<u>eno</u>	<u>pno</u>	resp	hours
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36

proj table

<u>pno</u>	pname	budget
P1	Instruments	150000
P2	DB Develop	135000
P3	Budget	250000
P4	Maintenance	310000
P5	CAD/CAM	500000

Subqueries in FROM Clause

Subqueries are used in the FROM clause to produce a derived table that can be used by the outer query.

Example: Return the departments that have an employee that makes more than \$40,000.

```
SELECT DISTINCT dname
FROM   dept D JOIN
      (SELECT dno FROM emp
       WHERE salary > 40000) AS E
      ON D.dno = E.dno
```

- Note: The alias for the derived table is required.

SQL Querying with Subqueries

Question: What query below is equivalent to:

```
SELECT  ename
FROM    emp as E
WHERE   salary > ALL (SELECT salary
                     FROM emp WHERE title = 'EE')
```

Note: Assume subquery returns at least one non-NULL salary value.

A)

```
SELECT  ename
FROM    emp as E
WHERE   salary > (SELECT MAX(salary) FROM emp
                 WHERE title = 'EE')
```

B)

```
SELECT  ename
FROM    emp as E
WHERE   salary > (SELECT SUM(salary) FROM emp
                 WHERE title = 'EE')
```

SQL Functions

Databases have many built-in functions that can be used when writing queries. Syntax and support varies between systems.

- **Date:** YEAR, DATEDIFF, CURRENT_DATE
- **String:** CONCAT, UPPER, LEFT, SUBSTRING
- **Logical:** CASE, IF
- **Aggregate:** SUM, COUNT, AVG, MIN, MAX
- **Note:** Case-insensitive function names.

Example:

```
SELECT eno, UPPER(ename),  
       CASE title WHEN 'EE' THEN 'Engineer'  
       WHEN 'SA' THEN 'Admin' ELSE 'Other' END as role,  
       year(bdate) as birthYear  
FROM emp  
WHERE salary * 2 > 60000
```

Try-it: Subquery Practice Questions

Question: Write these queries:

- 1) List all department names that have at least one project.
- 2) List the employee names who are not working on any project.
- 3) List the employee names with title 'EE' that make more than all employees with title 'PR'. Note: SQLite (PrairieLearn) does not support `> ALL`.
- 4) Find all employees who work on some project that 'J. Doe' works on. Return the employee name. Do not have 'J. Doe' as an answer.



SQL Queries using SELECT

A query in SQL has the form:

SELECT (list of columns or expressions)

FROM (list of tables)

WHERE (filter *conditions*)

GROUP BY (columns)

HAVING (group filter *conditions*)

ORDER BY (columns)

LIMIT (count) **OFFSET** (start)

Conclusion

SQL queries can summarize and combine data in several ways:

- Aggregate functions summarize sets of rows.
- GROUP BY calculates aggregates for groups of rows.
- HAVING filters groups after aggregation.
- Subqueries use the result of one query within another.
- Outer joins preserve rows that do not have a match.

There are often multiple SQL formulations for the same task.

Objectives

- Write SQL queries using aggregate functions, GROUP BY, and HAVING.
- Write scalar and table subqueries using comparison operators, IN, NOT IN, ANY/ALL, and EXISTS and NOT EXISTS.
- Explain and use correlated subqueries.
- Explain and use natural and outer joins.
- Find and use DBMS-specific SQL functions from documentation.



THE UNIVERSITY OF BRITISH COLUMBIA

